```
     Microprocessor-Lab
        Assignment

Name --> Akhilesh Kumar
Enroll --> 2022BITE029


Part 1: Solution

1. 4-bit Equality Comparator
The comparator checks if two 4-bit inputs
A and B are equal.

module EqualityComparator(
    input [3:0] A,     // 4-bit input A
    input [3:0] B,     // 4-bit input B
    output Equal       // Output, 1 if A == B
);
    // Check if all bits are equal
    assign Equal = (A == B) ? 1'b1 : 1'b0;
endmodule


2. 4-bit Adder
A 4-bit adder with a carry-out.

module FourBitAdder(
    input [3:0] A,     // 4-bit input A
    input [3:0] B,     // 4-bit input B
    input Cin,         // Carry input
    output [3:0] Sum,  // 4-bit Sum output
    output Cout        // Carry output
);
    assign {Cout, Sum} = A + B + Cin;
endmodule


3. 4-bit Multiplier
A simple implementation of a 4-bit multiplier using combinational logic.

module FourBitMultiplier(
    input [3:0] A,     // 4-bit input A
    input [3:0] B,     // 4-bit input B
    output [7:0] Product  // 8-bit Product output
);
    assign Product = A * B;
endmodule


4. 4-bit Divider
A basic unsigned divider for 4-bit inputs.

module FourBitDivider(
    input [3:0] Dividend,  // 4-bit Dividend
    input [3:0] Divisor,   // 4-bit Divisor
    output [3:0] Quotient, // 4-bit Quotient
    output [3:0] Remainder // 4-bit Remainder
);
    assign Quotient = Dividend / Divisor;
    assign Remainder = Dividend % Divisor;
```

```
endmodule


Part 2: 8-bit ALU

module ALU(
    input [7:0] A,     // 8-bit input A
    input [7:0] B,     // 8-bit input B
    input [2:0] ALUOp, // 3-bit ALU operation selector
    output reg [7:0] Result, // 8-bit result
    output Zero       // Zero flag
);

    always @(*) begin
        case (ALUOp)
            3'b000: Result = A + B;    // Addition
            3'b001: Result = A - B;    // Subtraction
            3'b010: Result = A & B;    // AND
            3'b011: Result = A | B;    // OR
            3'b100: Result = A ^ B;    // XOR
            3'b101: Result = ~(A | B); // NOR
            default: Result = 8'b00000000; // Default to zero
        endcase
    end

    // Zero flag is set when the result is zero
    assign Zero = (Result == 8'b00000000) ? 1'b1 : 1'b0;
endmodule


Part 3: In-Order Processor Design

Processor Design Outline
Instruction Fetch: Get the instruction to execute.
Instruction Decode: Decode the instruction to identify the operation and operands.
Execute: Perform the operation using the ALU.
Write Back: Store the result.
Instruction Set Architecture (ISA)
We can define a simple 8-bit instruction format:

[Opcode (3 bits)] [Operand1 (3 bits)] [Operand2 (3 bits)]
Opcode:
000: Add
001: Subtract
010: AND
011: OR
100: XOR
101: NOR

module Processor(
    input clk,                  // Clock signal
    input reset,                // Reset signal
    input [7:0] instruction,    // 8-bit instruction input
    input [7:0] dataA,          // Input data A
    input [7:0] dataB,          // Input data B
    output reg [7:0] result,    // 8-bit output result
    output reg zero             // Zero flag output
);

    // ALU operation selector
```

```
    reg [2:0] ALUOp;
    wire [7:0] ALUResult;
    wire ZeroFlag;

    // ALU Instance
    ALU alu_instance (
        .A(dataA),
        .B(dataB),
        .ALUOp(ALUOp),
        .Result(ALUResult),
        .Zero(ZeroFlag)
    );

    // Opcode extraction from instruction
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            result <= 8'b0;
            zero <= 1'b0;
        end else begin
            // Extract opcode from instruction
            ALUOp <= instruction[7:5]; // 3 MSBs for Opcode

            // Execute ALU operation
            result <= ALUResult;
            zero <= ZeroFlag;
        end
    end
endmodule


```